

Run IDs in the LQABR Email Agent

A plain-language reference — what a run ID is, why you see several, whether they're needed, and how they're created and attached.

1. Why am I seeing multiple run IDs for one lead?

Because **each outreach run for a lead gets its own freshly generated ID**, and this lead has been through several of them. A run ID marks one execution that emailed the lead — one send, one ID.

When you check a lead's status, **refresh_lead_status** calls **runs_for_object()**, which scans the run-state directory and returns every run file that contains this lead, then pulls each one. So if the lead sits in five run files, you see five run IDs:

- **d04fb3d5...** — the send you just made now
- **9dd33668...** — an earlier send (the one that had the legacy issue)
- **6b6e2ad2...** — an earlier send
- **c9c23667...** — an earlier send
- **210d8a17...** — a manual (by-hand) send

So the multiple IDs mean the contact was worked in several separate runs — it was emailed/processed multiple times, each stamped with its own ID.

Why more than once? This is the duplicate-send gap: the lead selection query picks leads by trigger id but never excludes leads already marked **SENT**, so re-triggering the campaign re-picks the same lead and starts a new run (plus the one manual send). Each run keeps a distinct ID on purpose, so their engagement events never collide.

2. Is the run ID necessary?

The run ID itself: yes. It's how a delivered / opened / clicked event that arrives hours or days later gets matched back to the exact send it belongs to (the correlation token is `object_id:run_id`). Without it, Mailgun events couldn't be attributed to the right lead and run, and you'd lose open/click tracking entirely. The concept has to stay.

Seeing several run IDs for one lead: no. In a correct flow this lead would have one run ID (one send). The extra ones are a side effect of the duplicate-send gap, not a feature. The fix is to exclude already-**SENT** leads from selection — that stops new duplicate runs going forward, though it won't remove the historical ones already on disk.

3. How is a run ID generated and attached?

Generated — once, at run start

When a send begins, **start_run** calls **bind_run**, which mints a random **UUID4 hex string** (32 characters, e.g. c9c23667...9724d8e). A caller can pass an explicit run_id for replay/resume, but normally a fresh one is generated. It lives on the run's context for the whole run.

```
run_id = run_id or uuid.uuid4().hex
```

It becomes half of the **correlation token**: <object_id>:<run_id> — the key everything else uses.

Attached — in three places

Where	How & why
On the outbound email	Added as Mailgun user variables (correlation token, run id, object id). Mailgun echoes them back on every delivered/opened/clicked event.
In the run-state file	The run id is both the filename (object__run.json) and stored inside the JSON, so a late event can still find the send days later.
In the logs	All observability streams are keyed by object_id + run_id.

How it comes back

When a Mailgun event arrives (webhook push or a status pull), the echoed correlation token is split back into object_id and run_id, and the store loads that exact run's file and finds the lead.

In one line: generated as a UUID4 at run start, carried on the run context, stamped onto the email + the run-state file + the logs, and used later to match returning events back to the right lead and send.